[CSED233] Data Structure

One-day C++

Seung-Hwan Baek

POSTECH

Review: Data Structure

- Programs solve problems by processing information

Program = Data structure + Algorithm

- Data structure is about how information is organized
- Algorithm is about how programs carry out tasks

Review: Assignments

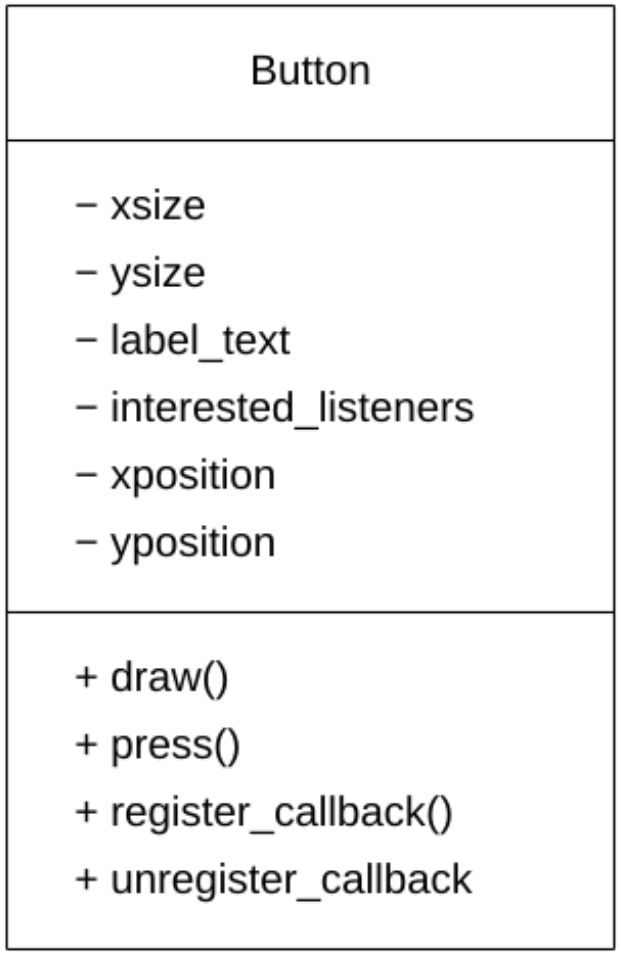
- Four programming assignments in C++
- Exploit the concepts taught in the classes
- You can use AI coding models
- Submit though PLMS
- Do not share your code with other students

What is C++?

- Extension to the C language
- Cross-platform language (Windows, Mac, Linux, …) that can be used to create high-performance applications (Anything that runs fast)
- High level of control over system resources and memory
- The major C++ versions: C++11, C++14, C++17, C++20, C++23 (latest)

Why C++?

- C++ is an object-oriented programming (OOP) language which gives a clear structure, in the form of Class consisting of data and functions
- Allows code to be reused, lowering development costs
- These days, it becomes easier to code C++ programs thanks to AI coding agents

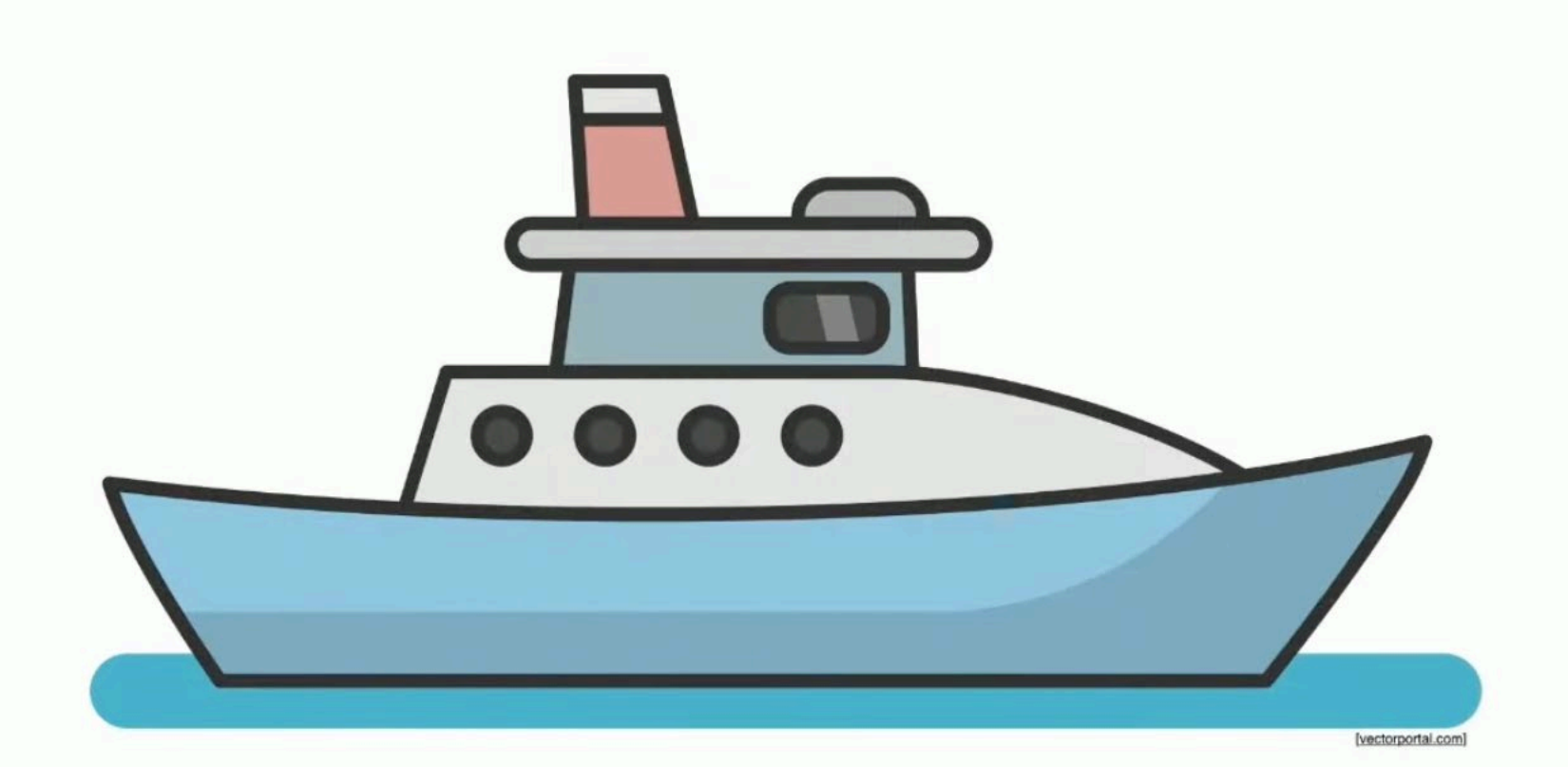


Examples

PyTorch

TensorFlow

Nice Boat

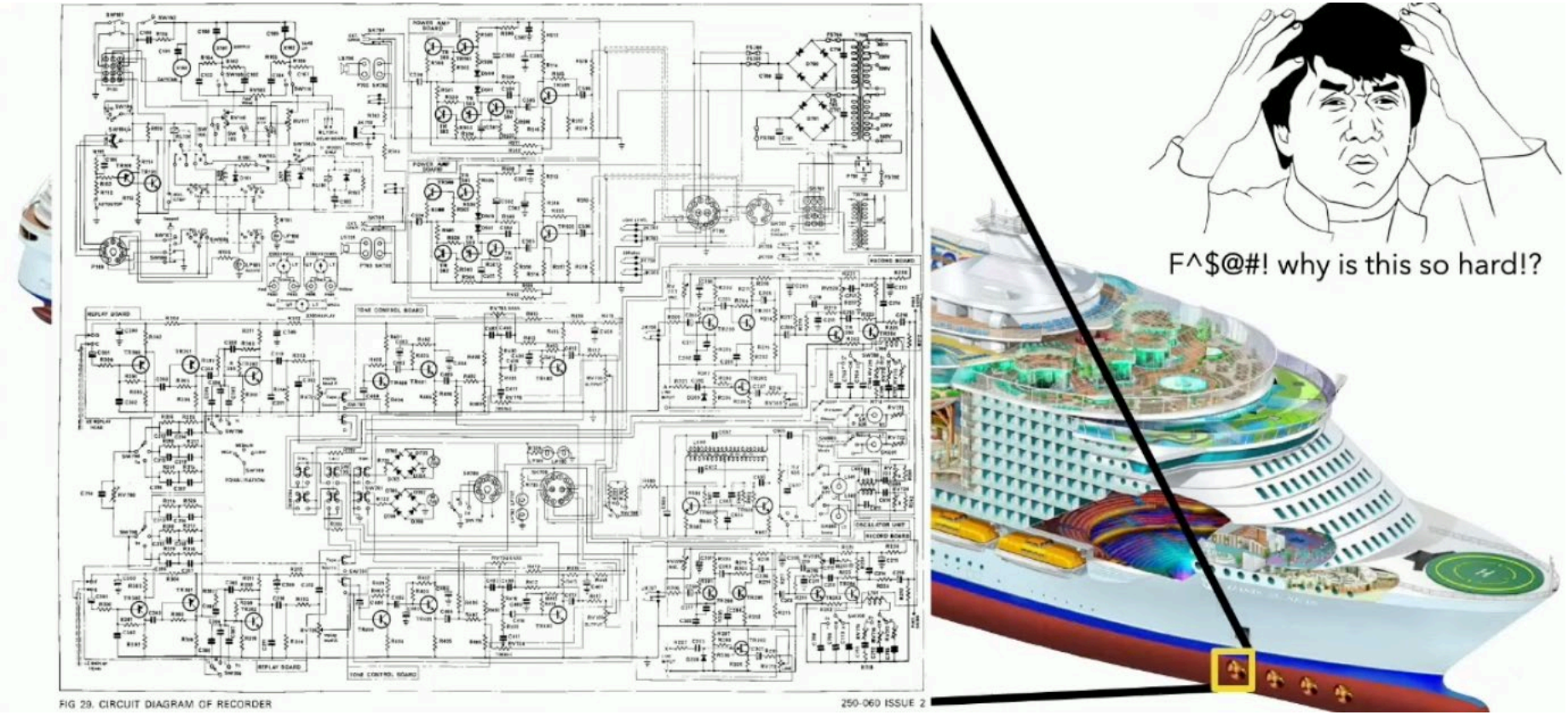


From Wenzel Jakob's Talk: https://www.youtube.com/watch?v=2KcrzVwP6-8

Upon Closer Inspection…

From Wenzel Jakob's Talk: https://www.youtube.com/watch?v=2KcrzVwP6-8

Upon Closer Inspection…



Could be C++!!!

From Wenzel Jakob's Talk: https://www.youtube.com/watch?v=2KcrzVwP6-8

In This Class

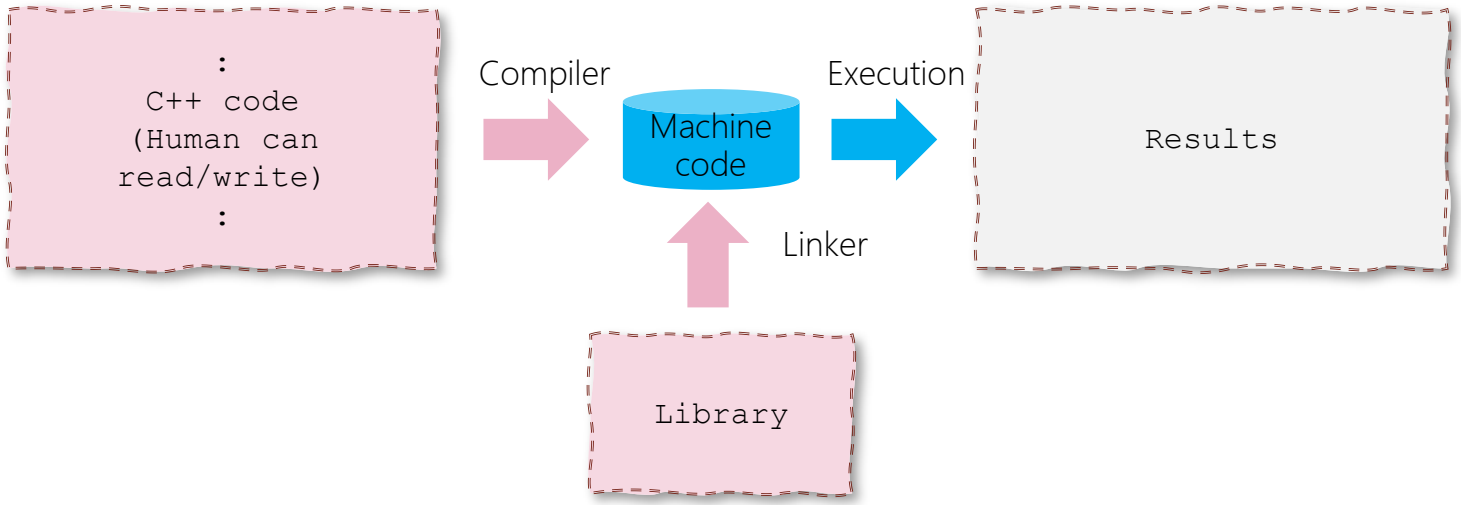
- Basic elements in C++
- Writing simple codes and see how it works
- Feel free to use AI coding agents!

```
:
C++ code
:
```

```
Result
```

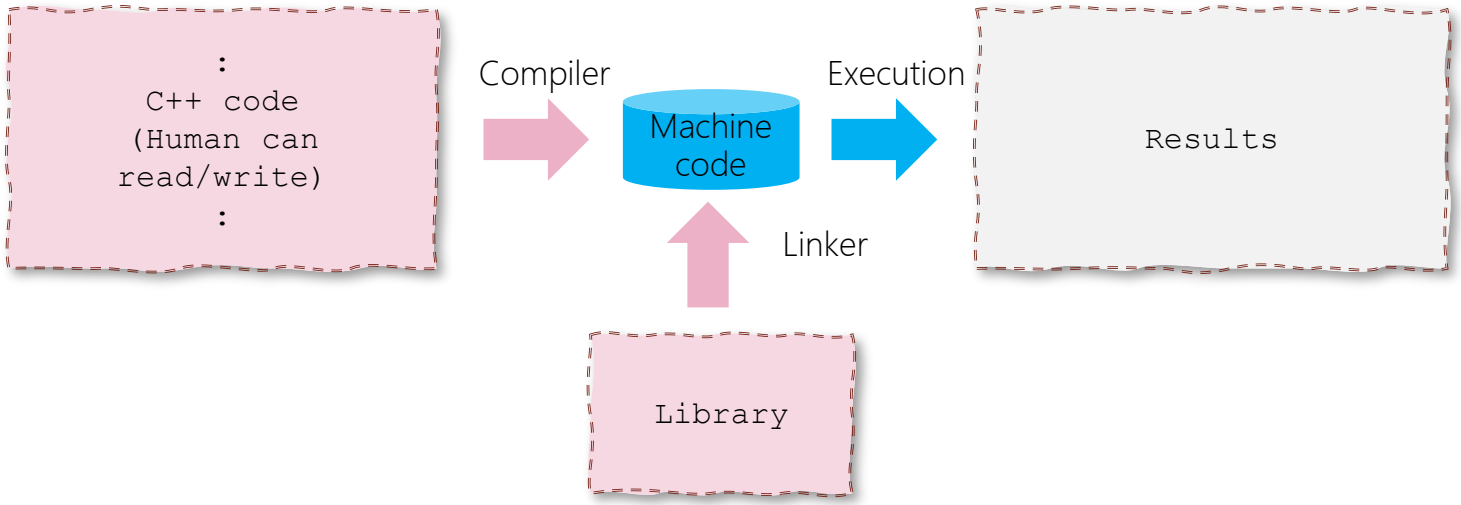
How C++ Works

1. **Create** a C++ source file and **save** it
2. **Compile**,
   - Using a *compiler*
     - creates a machine-code interpretation of your program
   - Using a *linker*
     - includes any required library functions needed
3. **Execute** program and see results



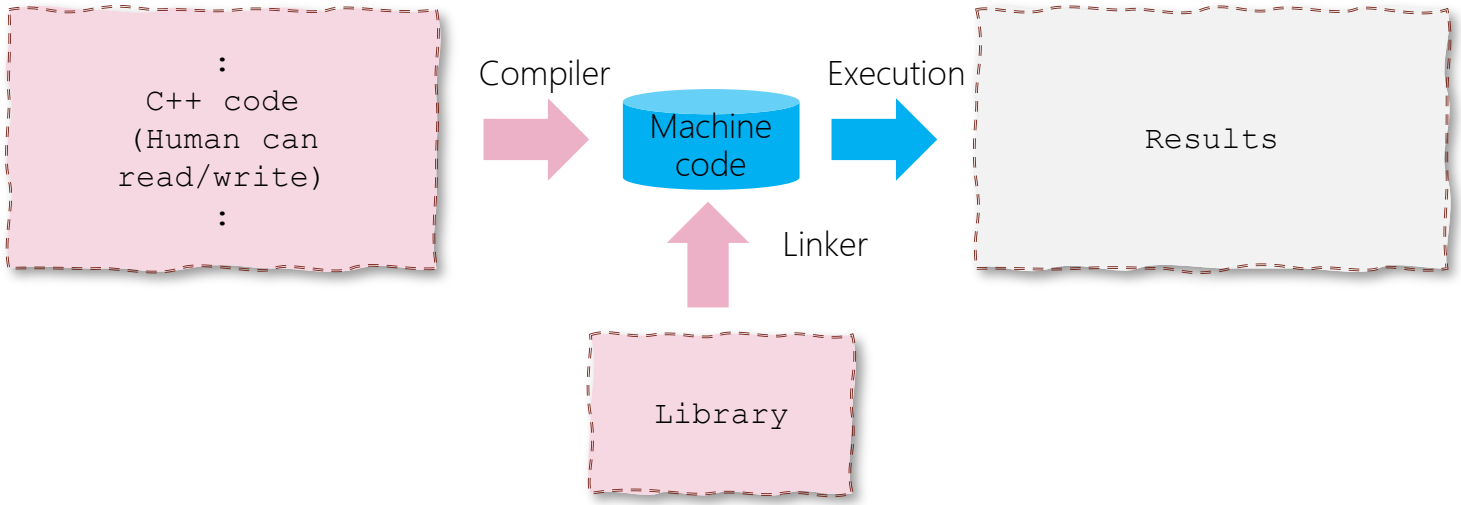
Quiz

- What are the inputs and outputs of the compiler?



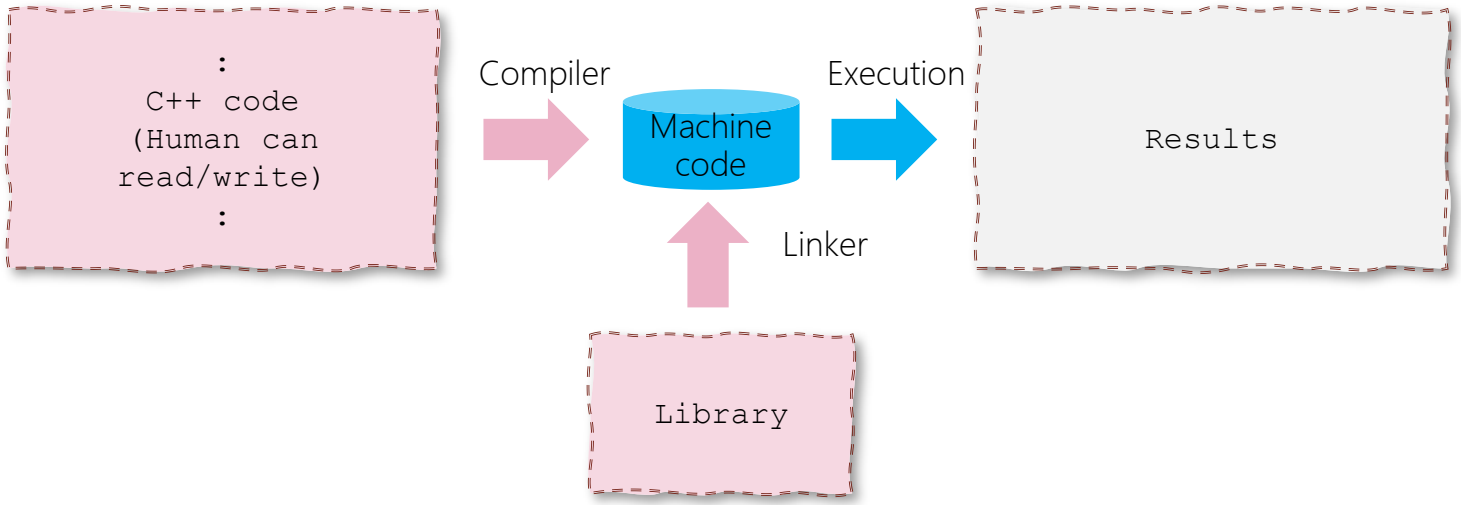
Quiz

- What are the inputs and outputs of the compiler?
  - Inputs: .cpp, .h
  - Outputs: .obj (windows), .o (linux) for each cpp file



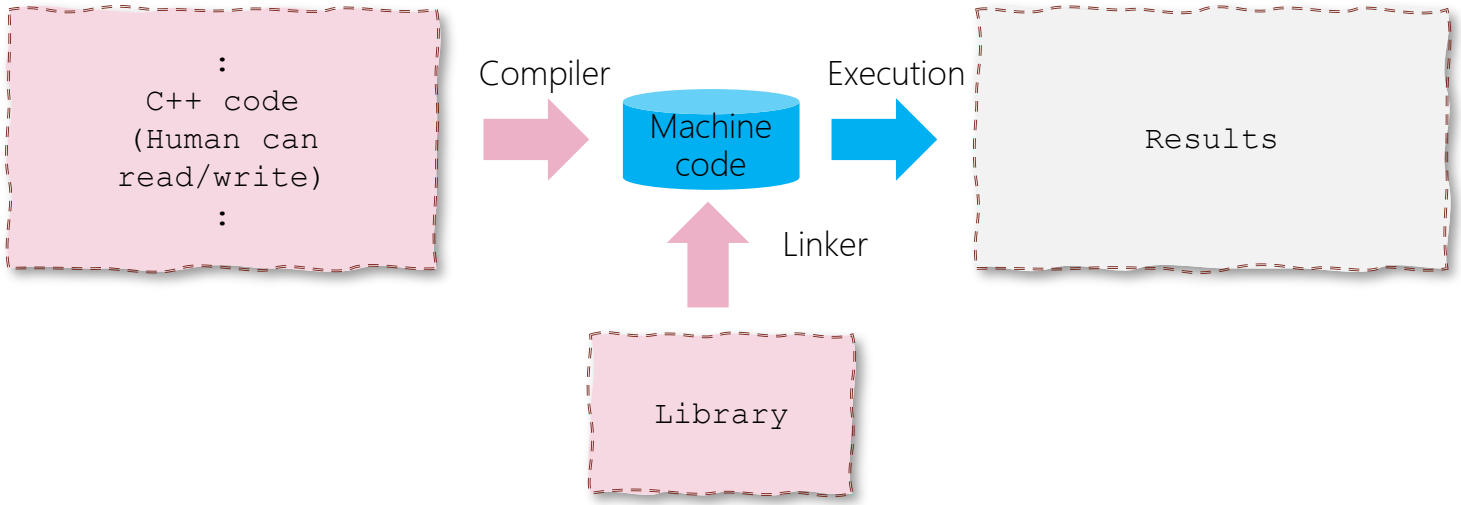
Quiz

- What are the inputs and outputs of the linker?



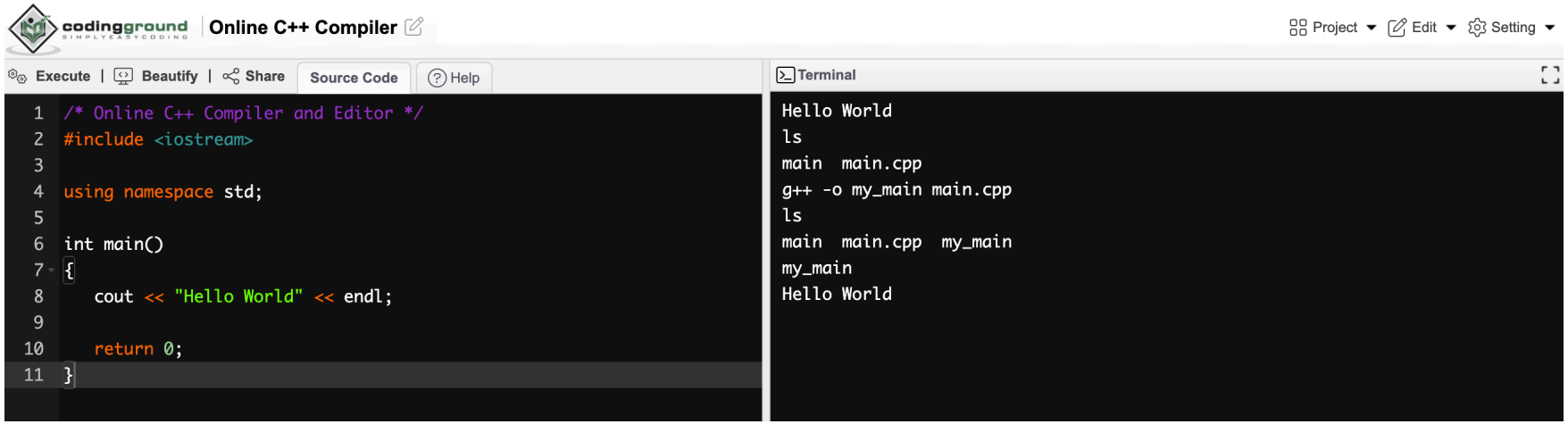
Quiz

- What are the inputs and outputs of the linker?
  - Inputs: multiple .o files
  - Outputs: .exe (windows), none (linux)



Try By Yourself: Basic C++ Example

- https://www.tutorialspoint.com/compile_cpp_online.php
- Click *Execute*
  - You will see Hello World on the terminal
- Type *ls* on the terminal to see file list you have
  - You will see *main* and *main.cpp*
- Type g++ -o my_main main.cpp
  - This converts *main.cpp* into a binary file (compiler + linker+ …)
  - Once you type ls again, you will see your binary file *my_main*
- Run your binary file by typing *my_main*
  - You will see Hello World



Simple C++ Program

```
>> Please enter two numbers: 7 35
Their sum is 42
```

Simple C++ Program

```
#include <cstdlib>
#include <iostream>
/* This program inputs two numbers x and y
   and outputs their sum */
int main( ) {
        int x, y;
        std::cout << "Please enter two numbers: ";
        std::cin >> x >> y;          // input x and y
        int sum = x + y;             // compute their sum
        std::cout << "Their sum is " << sum << std::endl;
        return 0;                    // terminate successfully
}
```

Header file

- #include <XX> is a header file library. Header files add functionality to C++ programs. You can make your own header files as well

```
#include <cstdlib>
#include <iostream>
/* This program inputs two numbers x and y
   and outputs their sum */
int main( ) {
        int x, y;
        std::cout << "Please enter two numbers: ";
        std::cin >> x >> y;          // input x and y
        int sum = x + y;             // compute their sum
        std::cout << "Their sum is " << sum << std::endl;
        return 0;                    // terminate successfully
}
```

Comments

- Comments can be used to explain C++ code, and to make it more readable. Comments can be singled-lined or multi-lined.

```
#include <cstdlib>
#include <iostream>
/* This program inputs two numbers x and y
   and outputs their sum */
int main( ) {
        int x, y;
        std::cout << "Please enter two numbers: ";
        std::cin >> x >> y;          // input x and y
        int sum = x + y;             // compute their sum
        std::cout << "Their sum is " << sum << std::endl;
        return 0;                    // terminate successfully
}
```

Main function

- Always appear in a C++ program: int main(). This is called a function. Any code inside its curly brackets {} will be executed.

```
#include <cstdlib>
#include <iostream>
/* This program inputs two numbers x and y
   and outputs their sum */
int main( ) {
        int x, y;
        std::cout << "Please enter two numbers: ";
        std::cin >> x >> y;          // input x and y
        int sum = x + y;             // compute their sum
        std::cout << "Their sum is " << sum << std::endl;
        return 0;                    // terminate successfully
}
```

Variable

- Variables are containers for storing data values.

```
#include <cstdlib>
#include <iostream>
/* This program inputs two numbers x and y
   and outputs their sum */
int main( ) {
        int x, y;
        std::cout << "Please enter two numbers: ";
        std::cin >> x >> y;          // input x and y
        int sum = x + y;             // compute their sum
        std::cout << "Their sum is " << sum << std::endl;
        return 0;                    // terminate successfully
}
```

Type

- In C++, there are different types of variables (defined with different keywords), for example:

- `bool` Boolean value, either true or false
- `char` character
- `short` short integer
- `int` integer
- `long long` long integer
- `float` single-precision floating-point number (float32)
- `double` double-precision floating-point number (float64)

Fundamental Types

Type	Size in bits	Format	Value range	
			Approximate	Exact
character	8	signed		**-128** to **127**
		unsigned		**0** to **255**
	16	unsigned		**0** to **65535**
	32	unsigned		**0** to **1114111** (**0x10ffff**)
integer	16	signed	**± 3.27 · 10^4**	**-32768** to **32767**
		unsigned	**0** to **6.55 · 10^4**	**0** to **65535**
	32	signed	**± 2.14 · 10^9**	**-2,147,483,648** to **2,147,483,647**
		unsigned	**0** to **4.29 · 10^9**	**0** to **4,294,967,295**
	64	signed	**± 9.22 · 10^{18}**	**-9,223,372,036,854,775,808** to **9,223,372,036,854,775,807**
		unsigned	**0** to **1.84 · 10^{19}**	**0** to **18,446,744,073,709,551,615**
floating point	32	IEEE-754	• min subnormal: **± 1.401,298,4 · 10^{-45}** • min normal: **± 1.175,494,3 · 10^{-38}** • max: **± 3.402,823,4 · 10^{38}**	• min subnormal: **±0x1p-149** • min normal: **±0x1p-126** • max: **±0x1.fffffep+127**
	64	IEEE-754	• min subnormal: **± 4.940,656,458,412 · 10^{-324}** • min normal: **± 2.225,073,858,507,201,4 · 10^{-308}** • max: **± 1.797,693,134,862,315,7 · 10^{308}**	• min subnormal: **±0x1p-1074** • min normal: **±0x1p-1022** • max: **±0x1.fffffffffffffp+1023**

From https://en.cppreference.com/w/cpp/language/types

Quiz

When would you use float64 over float32?

Type	Size in bits	Format	Value range	
			Approximate	Exact
character	8	signed		**-128** to **127**
		unsigned		**0** to **255**
	16	unsigned		**0** to **65535**
	32	unsigned		**0** to **1114111** (**0x10ffff**)
integer	16	signed	**± 3.27 · 10^4**	**-32768** to **32767**
		unsigned	**0** to **6.55 · 10^4**	**0** to **65535**
	32	signed	**± 2.14 · 10^9**	**-2,147,483,648** to **2,147,483,647**
		unsigned	**0** to **4.29 · 10^9**	**0** to **4,294,967,295**
	64	signed	**± 9.22 · 10^{18}**	**-9,223,372,036,854,775,808** to **9,223,372,036,854,775,807**
		unsigned	**0** to **1.84 · 10^{19}**	**0** to **18,446,744,073,709,551,615**
floating point	32	IEEE-754	• min subnormal: **± 1.401,298,4 · 10^{-45}** • min normal: **± 1.175,494,3 · 10^{-38}** • max: **± 3.402,823,4 · 10^{38}**	• min subnormal: **±0x1p-149** • min normal: **±0x1p-126** • max: **±0x1.fffffep+127**
	64	IEEE-754	• min subnormal: **± 4.940,656,458,412 · 10^{-324}** • min normal: **± 2.225,073,858,507,201,4 · 10^{-308}** • max: **± 1.797,693,134,862,315,7 · 10^{308}**	• min subnormal: **±0x1p-1074** • min normal: **±0x1p-1022** • max: **±0x1.fffffffffffffp+1023**

From https://en.cppreference.com/w/cpp/language/types

Quiz

When would you use float32 over float64?

Type	Size in bits	Format	Value range	
			Approximate	Exact
character	8	signed		**-128** to **127**
		unsigned		**0** to **255**
	16	unsigned		**0** to **65535**
	32	unsigned		**0** to **1114111** (**0x10ffff**)
integer	16	signed	**± 3.27 · 10^4**	**-32768** to **32767**
		unsigned	**0** to **6.55 · 10^4**	**0** to **65535**
	32	signed	**± 2.14 · 10^9**	**-2,147,483,648** to **2,147,483,647**
		unsigned	**0** to **4.29 · 10^9**	**0** to **4,294,967,295**
	64	signed	**± 9.22 · 10^{18}**	**-9,223,372,036,854,775,808** to **9,223,372,036,854,775,807**
		unsigned	**0** to **1.84 · 10^{19}**	**0** to **18,446,744,073,709,551,615**
floating point	32	IEEE-754	• min subnormal: **± 1.401,298,4 · 10^{-45}** • min normal: **± 1.175,494,3 · 10^{-38}** • max: **± 3.402,823,4 · 10^{38}**	• min subnormal: **±0x1p-149** • min normal: **±0x1p-126** • max: **±0x1.fffffep+127**
	64	IEEE-754	• min subnormal: **± 4.940,656,458,412 · 10^{-324}** • min normal: **± 2.225,073,858,507,201,4 · 10^{-308}** • max: **± 1.797,693,134,862,315,7 · 10^{308}**	• min subnormal: **±0x1p-1074** • min normal: **±0x1p-1022** • max: **±0x1.fffffffffffffp+1023**

From https://en.cppreference.com/w/cpp/language/types

Standard Functions

- The prefix std:: indicates that these objects are from the system's standard library. It is under "std" namespace. Those objects were included by the include <>

*A **namespace** is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc) inside it.

```
#include <cstdlib>
#include <iostream>
/* This program inputs two numbers x and y
   and outputs their sum */
int main( ) {
        int x, y;
        std::cout << "Please enter two numbers: ";
        std::cin >> x >> y;          // input x and y
        int sum = x + y;             // compute their sum
        std::cout << "Their sum is " << sum << std::endl;
        return 0;                    // terminate successfully
}
```

Inputs and Outputs

- The cout object, together with the << operator, is used to output values/print text
- cin is a predefined function that reads data from the keyboard with the extraction operator (>>).

```
#include <cstdlib>
#include <iostream>
/* This program inputs two numbers x and y
   and outputs their sum */
int main( ) {
        int x, y;
        std::cout << "Please enter two numbers: ";
        std::cin >> x >> y;          // input x and y
        int sum = x + y;             // compute their sum
        std::cout << "Their sum is " << sum << std::endl;
        return 0;                    // terminate successfully
}
```

Quiz

- You wrote a function named cout

```
void cout(const std::string& s)
{
        std::fprintf(stdout, "[mine] %s", s.c_str());
}
```

- How do you call your function and the standard one in the same program?

Enumeration

- A user-defined type
  - that can hold any of a set of discrete values
  - Once defined, enumerations behave much like an integer type

```
enum Day { SUN, MON, TUE, WED, THU, FRI, SAT };
enum Mood { HAPPY = 3, SAD = 1, ANXIOUS = 4, SLEEPY = 2 };

Day today = THU;            // today may be any of MON ... SAT
Mood myMood = SLEEPY;       // myMood may be HAPPY, ..., SLEEPY
```

Address and Value

Computer		Programmers		
Address	**Content**	**Name**	**Type**	**Value**
90000000	00	sum	int (4 bytes)	000000FF(255_{10})
90000001	00			
90000002	00			
90000003	FF			
90000004	FF	age	short (2 bytes)	FFFF(-1_{10})
90000005	FF			
90000006	1F	averge	double (8 bytes)	1FFFFFFFFFFFFFFF ($4.45015E-308_{10}$)
90000007	FF			
90000008	FF			
90000009	FF			
9000000A	FF			
9000000B	FF			
9000000C	FF			
9000000D	FF			
9000000E	90	ptrSum	int* (4 bytes)	90000000
9000000F	00			
90000010	00			
90000011	00			

Note: All numbers in hexadecimal

https://www3.ntu.edu.sg/home/ehchua/programming/cpp/cp4_PointerReference.html

Pointers

- Each variable is stored in the computer's memory at some location
  - It is called address
  - A pointer is a variable that holds the value of such an address.
    - Given a type `T`, the type `T*` denotes a pointer to a variable of type `T`.

Pointers

- Essential operators are used to manipulate pointers.
  - Address-of operator &
    - For example, if x is an integer variable in your program &x is the address of x in memory.
  - Dereferencing operator *
    - For example,
      - if we were to declare q to be a pointer to an integer (that is, int*)
      - and then set q = &x, we could access x's value with *q.
      - Assigning an integer value to *q effectively changes the value of x.

Pointer Example

```
char ch = 'Q';
char* p = &ch;          // p holds the address of ch
cout << *p;             // outputs the character 'Q'
ch = 'Z';               // ch now holds 'Z'
cout << *p;             // outputs the character 'Z'
*p = 'X';               // ch now holds 'X'
cout << ch;             // outputs the character 'X'
```

Array

- An *array* is a collection of elements of the same type
  - Given any type T and a constant *N*, a variable of type T[N] holds an array of *N* elements, each of type T
  - Each element of the array is referenced by its *index*,
    - that is, a number from 0 to *N* – 1.

```
double f[5];          // array of 5 doubles: f[0], . . ., f[4]
int m[10];            // array of 10 ints: m[0], ..., m[9]
f[4] = 2.5;
m[2] = 4;
cout << f[m[2]];      // outputs f[4], which is 2.5
```

Arrays

- When declaring an array,
  - We can initialize its values by enclosing the elements in curly braces ({…})
  - When doing so, we do not have to specify the size of the array
    - the **compiler** can figure this out

```
int a[] = {10, 11, 12, 13};        // declares and initializes a[4]
bool b[] = {false, true};          // declares and initializes b[2]
char c[] = {'c', 'a', 't'};        // declares and initializes c[3]
```

Pointers and Arrays

- Interesting connection between arrays and pointers
  - the name of an array is equivalent to a pointer to the array's initial element and vice versa
  - In the example below, c is an array of characters, and p and q are pointers to the first element of c. They all behave essentially the same

```
char c[] = {'c', 'a', 't'};
char* p = c;                    // p points to c[0]
char* q = &c[0];                // q also points to c[0]
cout << c[2] << p[2] << q[2];   // outputs "ttt"
```

String

- C++ provides a string type in Standard Template Library (STL)
  - In order to use STL strings it is necessary to include the header file <string>
  - Since STL strings are part of the standard namespace
  - By adding the statement "using std::string," we inform the compiler that we want to access this definition directly, so we can omit the "std::" prefix
  - Lexicographical comparison based on the ASCII code
    - From the left to the right

```
#include <string>
using std::string;
// ...
string s = "to be";
string t ="not" + s;          // t = "not to be"
string u = s + "or" + t;      // u = "to be or not to be"
if (s > t)                    // true: "to be" > "not to be"
       cout << u;             // outputs "to be or not to be"
```

String

- We can append one string to another using the += operator
- Also, strings may be indexed like arrays
- The number of characters in a string s is given by s.size()

```
string s = "John";          // s = “John”
int i = s.size();           // i = 4
char c = s[3];              // c = ’n’
s += " Smith";              // now s = “John Smith”v
```

Structure

- A *structure*
  - useful for storing an aggregation of elements
  - Unlike an array, the elements of a structure may be of different types
  - Each *member*, or *field*, of a structure is referred to by a given name.

```
enum MealType { NO PREF, REGULAR, LOW FAT, VEGETARIAN };

struct Passenger {
    string     name;          // passenger name
    MealType   mealPref;      // meal preference
    Bool       isFreqFlyer;   // in the frequent flyer program?
    string     freqFlyerNo;   // the passenger's freq. flyer number
};

// Declare and initialize a variable named "pass" of this type.
Passenger pass = { "John Smith", VEGETARIAN, true, "293145" };
```

Dynamic memory

- Data structures to create objects dynamically as the need arises
  - The C++ run-time system reserves a large block of memory called the *free store* or *heap memory*
  - The operator *new* dynamically allocates the correct amount of storage
    - For an object of a given type from the free store and returns a pointer to this object
    - That is, the value of this pointer is the address where this object resides in memory

```
Passenger *p;
// ...
p = new Passenger;          // p points to the new Passenger
p->name = "Pocahontas";     // set the structure members
p->mealPref = REGULAR;
p->isFreqFlyer = false;
p->freqFlyerNo = "NONE";
```

Dynamic memory

- C++ does not provide automatic *garbage collection*.
  - This means that C++ programmers have the responsibility of explicitly deleting all dynamically allocated objects.
- Failure to delete dynamically allocated objects can cause problems
  - If we were to change the (address) value of p without first deleting the structure to which it points, there would be no way for us to access this object
  - Having such inaccessible objects in dynamic memory is called a *memory leak*.

```
delete p;       // destroy the object p points to
```

Dynamic memory

- Arrays can also be allocated with new.
  - The system allocator returns a pointer to the first element of the array
  - A dynamically allocated array with elements of type T would be declared being of type *T
  - Cannot be deallocated using the standard delete operator.
    - Instead, the operator **delete[ ]** is used.

```
char* buffer = new char[500];      // allocate a buffer of 500 chars
buffer[3] = 'a';                   // elements are still accessed using []
delete [] buffer;                  // delete the buffer
```

References

- Pointers provide one way to refer indirectly to an object
  - A *reference* is simply an alternative name for an object
    - Given a type T, the notation T& indicates a reference to an object of type T.
    - Unlike pointers, which can be NULL, a reference in C++ must refer to an actual variable.
    - When a reference is declared, its value must be initialized.
    - Afterwards, any access to the reference is treated exactly as if it is an access to the underlying object.

```
string author = "Samuel Clemens";
string& penName = author;          // penName is an alias for author
penName = "Mark Twain";            // now author = “Mark Twain”
cout << author;                    // outputs “Mark Twain”
```

Operator Precedences

Type	*Operators*
scope resolution	namespace_name :: member
selection/subscripting function call postfix operators	class_name.member pointer->member array[exp] function(args) var++ var--
prefix operators dereference/address	++var --var +exp -exp ~exp !exp *pointer &var
multiplication/division	* / %
addition/subtraction	+ -
shift	<< >>
comparison	< <= > >=
equality	== !=
bitwise and	&
bitwise exclusive-or	^
bitwise or	|
logical and	&&
logical or	||
conditional	bool_exp ? true_exp : false_exp
assignment	= += -= *= /= %= >>= <<= &= ^= |=

Expressions

- Member Selection and Indexing

```
class name.Member            // class/structure member selection
pointer->member array        // class/structure member selection
array[ exp ]                 // array subscripting
```

Arithmetic Operators

```
int exp = 3;
int rst = 0;
rst = exp + exp      // addition
rst = exp - exp      // subtraction
rst = exp * exp      // multiplication
rst = exp / exp      // division
rst = exp % exp      // modulo (remainder)
```

Increment

- The *post-increment* operator
  - returns a variable's value and then increments it by 1.
- The *pre-increment* operator
  - first increments the variables and then returns the value.

```
var++          // post increment
var--          // post decrement
++var          // pre increment
--var          // pre decrement

int a[] = {0, 1, 2, 3};
int i = 2;
int j = i++;          // j = 2 and now i = 3
int k = --i;          // now i = 2 and k = 2
cout << a[k++];       // a[2] (= 2) is output; now k = 3
```

Relational and Logical Operators

- C++ provides the usual comparison operators.
- These return a Boolean result—either true or false. Comparisons can be made between numbers, characters, and STL strings

```
exp < exp              // less than
exp > exp              // greater than
exp <= exp             // less than or equal
exp >= exp             // greater than or equal
exp == exp             // equal to
exp != exp             // not equal to

!exp                   // logical not
exp && exp             // logical and
exp || exp             // logical or
```

Casting

- Casting is an operation that allows us to change the type of a variable.
- Casting to a type of higher precision or size is often needed in forming expressions.
  - For example, division between integers always produces an integer result by truncating the fractional part.

```
int cat = 14;
double dog = (double) cat;          // traditional C-style cast
double pig = double(cat);           // C++ functional cast

int i1 = 18;
int i2 = 16;
double dv1 = i1 / i2;                       // dv1 has value 1.0
double dv2 = double(i1) / double(i2);       // dv2 has value 1.125
double dv3 = double( i1 / i2 );             // dv3 has value 1.0
```

Implicit casting

- There are many instances where the programmer has not requested an *explicit cast*, but a change of types is required.
  - For example, when numbers of different types are involved in an operation, the compiler automatically casts to the stronger type.
  - C++ allows an assignment that implicitly loses information, but the compiler usually issues a warning message.

```
int i = 3;
double d = 4.8;
double d3 = i / d;    // d3 = 0.625 = double(i)/d
int i3 = d3;          // i3 = 0 = int(d3)
// Warning! Assignment may lose information
```

For Loop

- Many loops involve three common elements:
  - an initialization
  - a condition under which to continue execution
  - and an increment to be performed after each execution of the loop's body
- A *for loop* conveniently encapsulates these three elements.

```
for ( initialization ; condition ; increment )
        loop body statement
```

For Loop

- Here is a simple example, which prints the positive elements of an array, one per line
  - `i` was declared as **`int`** `i = 0`
  - Before each iteration, the loop tests the condition "`i < NUM ELEMENTS`" and executes the loop body only if this is true
  - At the end of each iteration the loop uses the statement `i++` to increment the loop variable `i` before testing the condition again

```
const int NUM ELEMENTS = 10;
double b[NUM ELEMENTS] = {1.34,2.58,-3.61,-4.60,-
5.03,5.23,4.38,3.34,-2.46,-1.56};

for (int i = 0; i < NUM ELEMENTS; i++) {
        if (b[i] > 0)
                cout << b[i] << '\n';
}
```

Break Statements

- C++ provides statements to change control flow
- A break statement is used to "break" out of a loop statement
  - When it is executed, it causes the flow of control to immediately exit the innermost loop.
- The following example:
  - Sums the elements of an array until finding the first negative value.

```
int a[10] = {1,2,-3,-4,-5,5,4,3,-2,-1};
int sum = 0;

for (int i = 0; i < 10; i++) {
    if (a[i] < 0) break;
    sum += a[i];
}
cout << "sum : " << sum << endl;
```

Break Statements

- The following example - equivalent
  - Sums the elements of an array until finding the first negative value.

```
int a[10] = {1,2,-3,-4,-5,5,4,3,-2,-1};
int sum = 0;

for (int i = 0; a[i] > 0 && i < 10; i++) {
    sum += a[i];
}

cout << "sum : " << sum << endl;
```

Continue Statement

- The continue statement can only be used inside loops
- The continue statement causes the execution to skip to the end of the loop, ready to start a new iteration

```
int a[10] = {1,2,-3,-4,-5,5,4,3,-2,-1};
int sum = 0;

for (int i = 0; i < 10; i++) {
    if (a[i] < 0) continue;
    sum += a[i];
}
cout << "sum : " << sum << endl;
```

Functions

- A *function* is a chunk of code that can be called to perform some well-defined task
  - Return type
    - This specifies the type of value or object that is returned by the function
  - Function name
    - This indicates the name that is given to the function
  - Argument list
    - This serves as a list of placeholders for the values that will be passed into the function
  - Function body
    - This is a collection of C++ statements that define the actual computations to be performed by the function

```
bool evenSum(int a[], int n) {                    // function declaration
        int sum = 0;
        for (int i = 0; i < n; i++)               // sum the array elements
                sum += a[i];
        return (sum % 2) == 0;                    // returns true if sum is even
}
```

Function declaration vs definition

- Function specifications in C++ typically involve two steps
  - A function is *declared*, by specifying three things: the function's return type, its name, and its argument list. This three-part combination is called the function's *signature* or *prototype*

```
bool evenSum(int a[], int n);            // function declaration
```

  - the function is *defined*. The definition consists both of the function's signature and the function body

```
bool evenSum(int a[], int n) {           // function declaration
        int sum = 0;
        for (int i = 0; i < n; i++)      // sum the array elements
                sum += a[i];
        return (sum % 2) == 0;           // returns true if sum is even
}
```

Argument Passing

- Sometimes it is useful for the function to modify one of its arguments. To do so, we can explicitly define a formal argument to be a *reference type*

```
void f (int value, int& ref ) {   // one value and one reference
        value++;                  // no effect on the actual argument
        ref ++;                   // modifies the actual argument

        cout << value << endl;    // outputs 2
        cout << ref << endl;      // outputs 6
}
int main() {
        int cat = 1;
        int dog = 5;
        f(cat, dog);              // pass cat by value, dog by ref
        cout << cat << endl;      // outputs 1
        cout << dog << endl;      // outputs 6
        return 0;
}
```

Function Overloading

- Two or more functions can be defined with the same name but with different argument lists
- Such definitions are useful in situations where we desire two functions that achieve essentially the same purpose, but do it with different types of arguments

```
void print(int x)                          // print an integer
{ cout << x; }

void print(const Passenger& pass) {        // print a Passenger
        cout << pass.name << " " << pass.mealPref;
        if (pass.isFreqFlyer)
                cout << " " << pass.freqFlyerNo;
}
```

Class

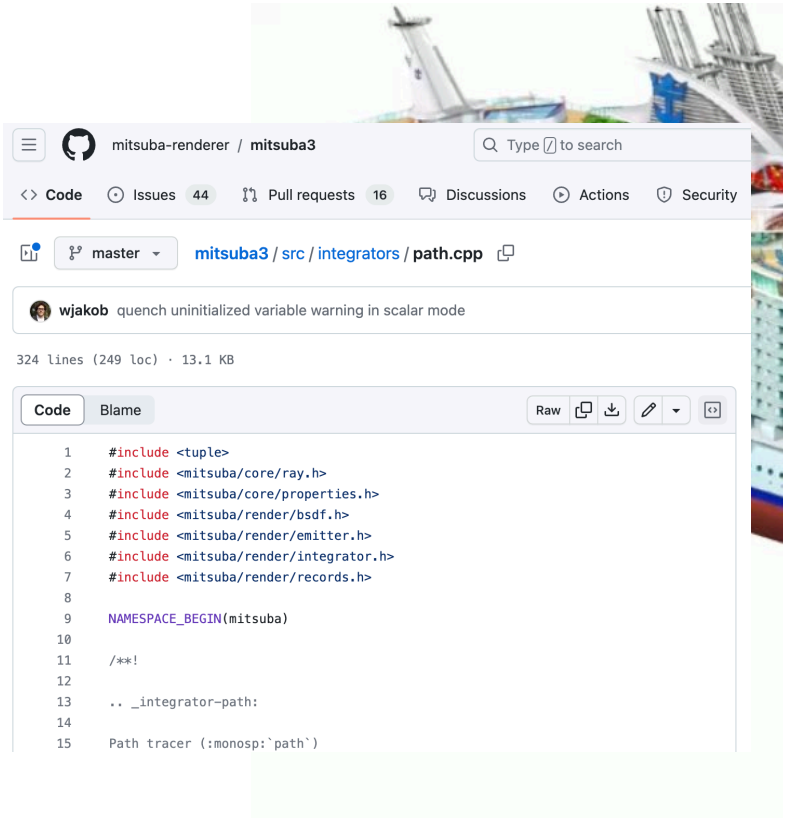
- Similar to Structure (normally used for grouping data)
- Class is often used for data abstraction and reuse
- There are more detailed differences about private, public, …

```
struct Passenger {
    string      name;
    MealType    mealPref;
    Bool        isFreqFlyer;
    string      freqFlyerNo;
};
```

```
class Passenger {
    string      name;
    MealType    mealPref;
    Bool        isFreqFlyer;
    string      freqFlyerNo;
};
```

What You will See In The Wild…

- Be prepared




```
87   template <typename Float, typename Spectrum>
88   class PathIntegrator : public MonteCarloIntegrator<Float, Spectrum> {
89   public:
90       MI_IMPORT_BASE(MonteCarloIntegrator, m_max_depth, m_rr_depth, m_hide_emitters)
91       MI_IMPORT_TYPES(Scene, Sampler, Medium, Emitter, EmitterPtr, BSDF, BSDFPtr)
92
93       PathIntegrator(const Properties &props) : Base(props) { }
94
95       std::pair<Spectrum, Bool> sample(const Scene *scene,
96                                        Sampler *sampler,
97                                        const RayDifferential3f &ray_,
98                                        const Medium * /* medium */,
99                                        Float * /* aovs */,
100                                       Bool active) const override {
101          MI_MASKED_FUNCTION(ProfilerPhase::SamplingIntegratorSample, active);
102
103          if (unlikely(m_max_depth == 0))
104              return { 0.f, false };
105
106          // --------------------- Configure loop state ----------------------
107
108          Ray3f ray                    = Ray3f(ray_);
109          Spectrum throughput          = 1.f;
110          Spectrum result              = 0.f;
111          Float eta                    = 1.f;
112          UInt32 depth                 = 0;
113
114          // If m_hide_emitters == false, the environment emitter will be visible
115          Mask valid_ray               = !m_hide_emitters && dr::neq(scene->environment(), nullptr);
116
117          // Variables caching information from the previous bounce
118          Interaction3f prev_si        = dr::zeros<Interaction3f>();
119          Float         prev_bsdf_pdf  = 1.f;
120          Bool          prev_bsdf_delta = true;
121          BSDFContext   bsdf_ctx;
122
123          /* Set up a Dr.Jit loop. This optimizes away to a normal loop in scalar
124             mode, and it generates either a a megakernel (default) or
125             wavefront-style renderer in JIT variants. This can be controlled by
126             passing the '-W' command line flag to the mitsuba binary or
127             enabling/disabling the JitFlag.LoopRecord bit in Dr.Jit.
128
129             The first argument identifies the loop by name, which is helpful for
130             debugging. The subsequent list registers all variables that encode
131             the loop state variables. This is crucial: omitting a variable may
132             lead to undefined behavior. */
133          dr::Loop<Bool> loop("Path Tracer", sampler, ray, throughput, result,
134                              eta, depth, valid_ray, prev_si, prev_bsdf_pdf,
135                              prev_bsdf_delta, active);
136
```

```
259              // Information about the current vertex needed by the next iteration
260              prev_si = si;
261              prev_bsdf_pdf = bsdf_sample.pdf;
262              prev_bsdf_delta = has_flag(bsdf_sample.sampled_type, BSDFFlags::Delta);
263
264              // -------------------- Stopping criterion ---------------------
265
266              dr::masked(depth, si.is_valid()) += 1;
267
268              Float throughput_max = dr::max(unpolarized_spectrum(throughput));
269
270              Float rr_prob = dr::minimum(throughput_max * dr::sqr(eta), .95f);
271              Mask rr_active = depth >= m_rr_depth,
272                   rr_continue = sampler->next_1d() < rr_prob;
273
274              /* Differentiable variants of the renderer require the the russian
275                 roulette sampling weight to be detached to avoid bias. This is a
276                 no-op in non-differentiable variants. */
277              throughput[rr_active] *= dr::rcp(dr::detach(rr_prob));
278
279              active = active_next && (!rr_active || rr_continue) &&
280                       dr::neq(throughput_max, 0.f);
281          }
282
283          return {
284              /* spec  = */ dr::select(valid_ray, result, 0.f),
285              /* valid = */ valid_ray
286          };
287      }
288
289      //! @}
290      // ============================================================
291
292      std::string to_string() const override {
293          return tfm::format("PathIntegrator[\n"
294              "  max_depth = %u,\n"
295              "  rr_depth = %u\n"
296              "]", m_max_depth, m_rr_depth);
297      }
298
299      /// Compute a multiple importance sampling weight using the power heuristic
300      Float mis_weight(Float pdf_a, Float pdf_b) const {
301          pdf_a *= pdf_a;
302          pdf_b *= pdf_b;
303          Float w = pdf_a / (pdf_a + pdf_b);
304          return dr::detach<true>(dr::select(dr::isfinite(w), w, 0.f));
305      }
306
307      /**
308       * \brief Perform a Mueller matrix multiplication in polarized modes, and a
309       * fused multiply-add otherwise.
310       */
311      Spectrum spec_fma(const Spectrum &a, const Spectrum &b,
312                        const Spectrum &c) const {
313          if constexpr (is_polarized_v<Spectrum>)
314              return a * b + c;
315          else
316              return dr::fmadd(a, b, c);
317      }
318
319      MI_DECLARE_CLASS()
320  };
321
322  MI_IMPLEMENT_CLASS_VARIANT(PathIntegrator, MonteCarloIntegrator)
323  MI_EXPORT_PLUGIN(PathIntegrator, "Path Tracer integrator");
324  NAMESPACE_END(mitsuba)
```

More About C++

- Much more than what we learned ☺
  - Template
  - Switch
  - Over-riding
  - Inheritance
  - …
- Learn from examples!
- If you become an expert in C++, you will also be able to write high-quality code in other programming languages!

C++ and Python

- Python
  - Easy to learn
  - Easy to acess libraries
  - Easy to test
- C++
  - Fast execution speed
  - Pre-compiled
- How to use both in a same project?

pybind11

Non-mandatory Homework

- Write a simple program that computes a sum (e.g., the sum of numbers from 1 to N) in both C++ and Python, and compare their execution times to observe the performance difference between the two languages
- You don't need to submit the results!
- Just try it for fun

Summary

- C++
- Compiler, linker
- Variable, pointer, array, dynamic memory, function, class, string, structure
- Header file, namespace, cin/cout, enumeration, reference
- Casting
- Loop, break, continue